SmartStock

AI-Powered Inventory & Demand Forecasting
Pharmaceutical & Warehouse Stock Management
Anesu | B.Eng. (Hons) Software & Electronic Engineering | ATU | 2025/2026

The Problem SmartStock Solves

Stockouts

Critical inventory items run out before reorders are placed, leaving customers without essential treatments and disrupting delivery.

Overstocking

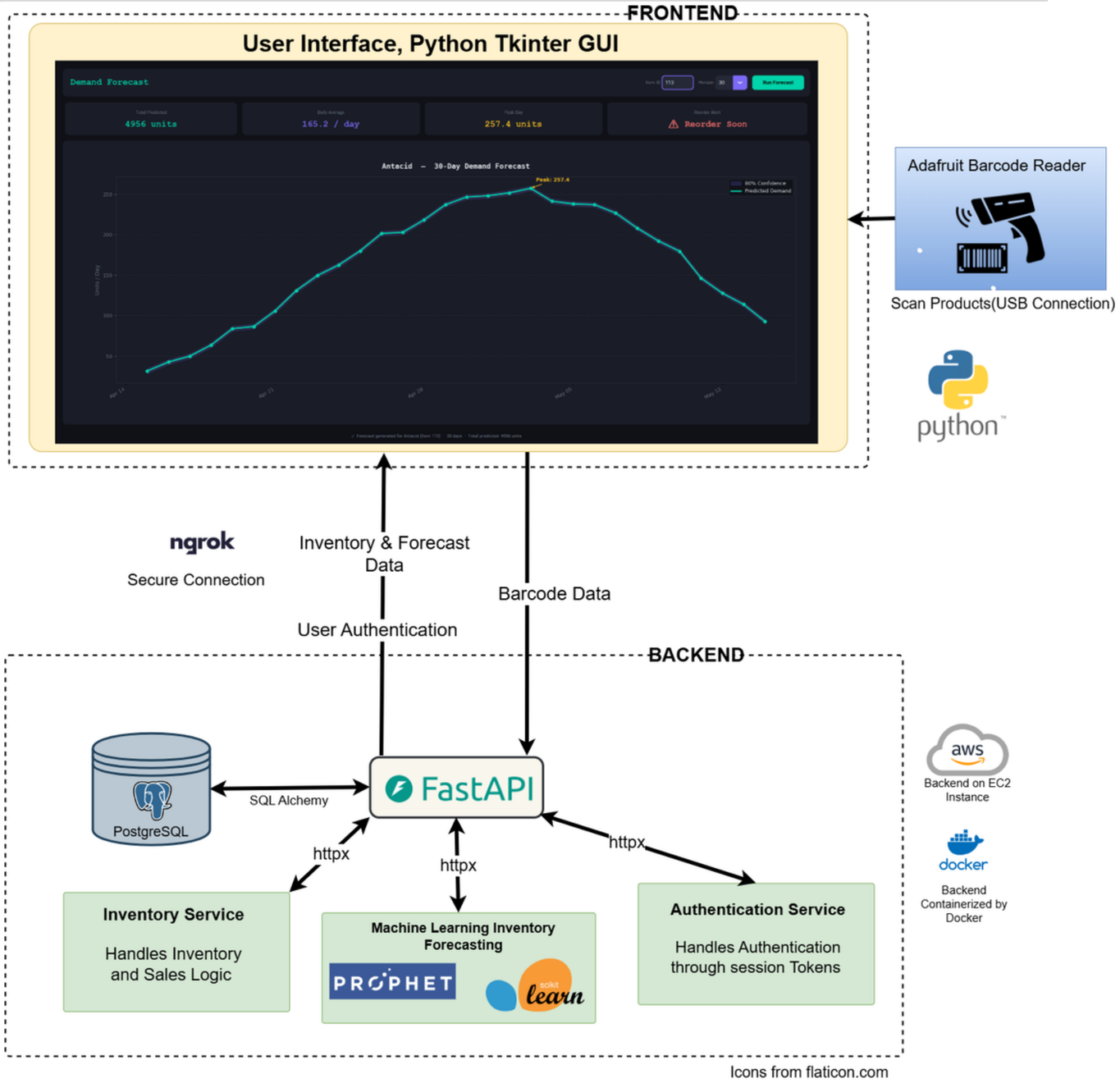
Items expire before they're used, wasting valuable budget and resources while contributing to waste and compliance issues.

No Forecasting

Manual systems can't predict when stockouts or overstocking will happen, leaving inventory managers reactive instead of proactive.

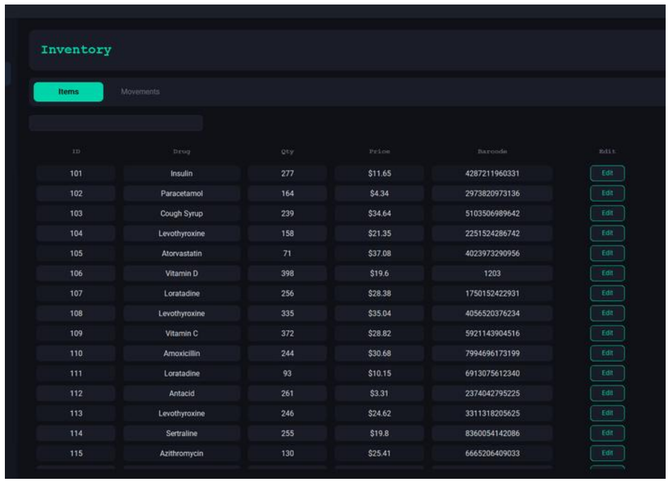
How SmartStock Is Built

Four-layer architecture ensuring separation of concerns, testability, and real-time data flow.



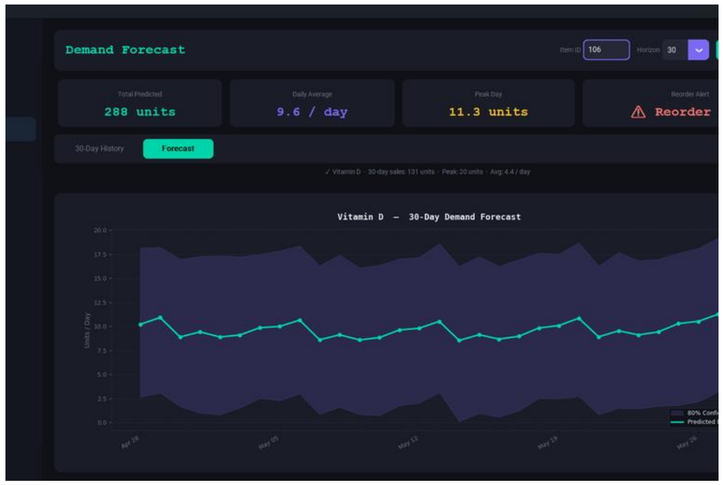
SmartStock in Action

Experience seamless inventory management with real-time updates and intelligent forecasting.



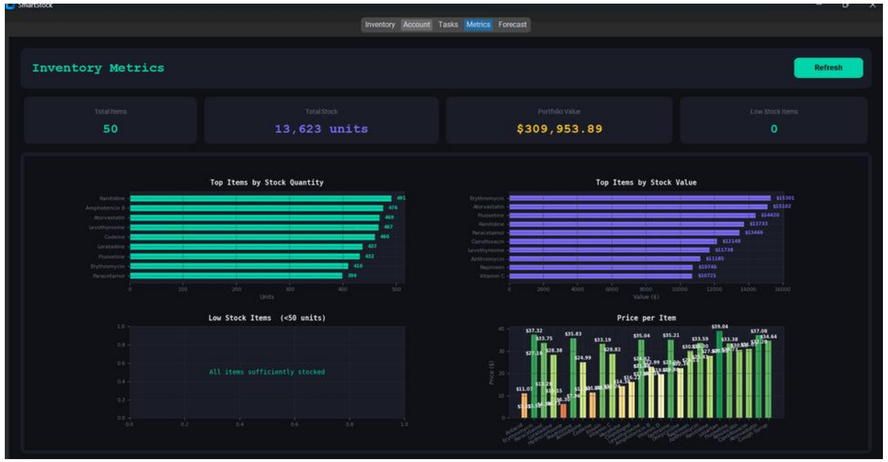
Inventory Tab

Item list, stock & expiry



Forecast Tab

Prophet chart & horizon



Real-Time Updates

Analyse Data

Two Models, Two Different Questions

Prophet (Statistical Forecasting)

- Fits curve to historical stock data
- Predicts future demand volume
- Answers: "How much will we need?"
- No labelled training data required

Random Forest (ML Classifier)

- Trained on labelled stockout examples
- Predicts stockout risk within 30 days
- Answers: "Which items are at risk?"
- Uses stock level, consumption velocity, expiry proximity

Testing Strategy

A comprehensive multi-layered approach ensuring SmartStock reliability from unit level to deployment.

Unit Tests

pytest with SQLite in-memory database for isolated, fast testing. Comprehensive coverage of all CRUD endpoints ensures reliability at the foundation level.

Integration Tests

Full HTTP request/response cycles using httpx TestClient. Validates complete API workflows and ensures all components work together seamlessly.

CI Pipeline & Manual

GitHub Actions runs pytest on every push to master for continuous quality assurance. Manual testing validates barcode scanner with live hardware and forecasting against seeded data.

How I Managed This Project

Agile project management tools and methodology used to deliver SmartStock systematically and on schedule.

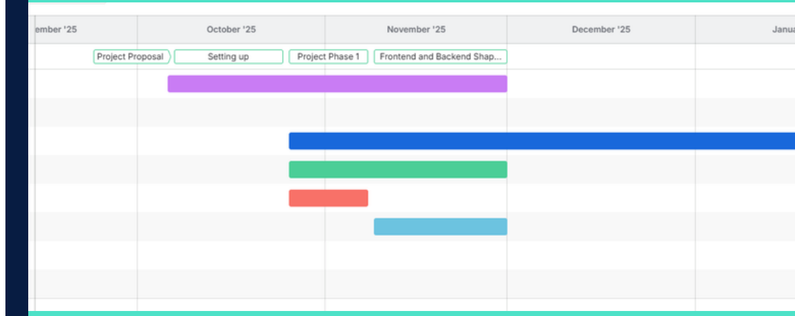
Scrum Methodology

Week 10 Standup

Tuesday 24 March 2026 10:17

	Last Week	This Week	Blockers
David	Built the action executor so now the agent can perform different actions on the hosts like blocking them, or restoring them to their original state. Started working on the Intrusion system so now the agent is aware of hosts marked as compromised	Working on changing the honeypot action to a different approach which is more realistic. Finish the intrusion system as I ran into an issue where the agent isn't switching phases so it's actions are very passive when it should be defensive	None
Anthony	Connected my frontend to my backend. Redesigning the flow of the proof generation.	Create a webpage frontend for third party apps to deploy a contract with the requirements. Users can generate a proof based on the requirements.	
Choon			
Naoise	Completed S3 integration and began planning next step	Integrate users, and begin on project poster/writeup	None
Nathan	Implemented LCOV Reports to create a HTML page on every push; showing all covered lines by test cases. Created UI for users to access daily challenges from previous days	Update Architect Diagram, begin working on poster.	None
Yi	finished frontend with all needed components displaying, but still need some time to design it. Started designing agentic workflow management and implemented vector embeddings.	finish up vector embeddings	None
Chinonso			
Me	Worked on integrating an NFC reader (X-NUCLEO-NFC12A1) into the system to enable tag-based security system. So the board was meant to be an arduino board and programmable through arduino, but I ran into compatibility issues with the official library designed for STM32 boards, so I have started building a low-level SPI interface instead. I also got a low-level authentication of Password and Username working on the Frontend	Next step is to confirm stable communication with the chip and also start writing test for my services.	None

Jira Tracking



OneNote Logbook

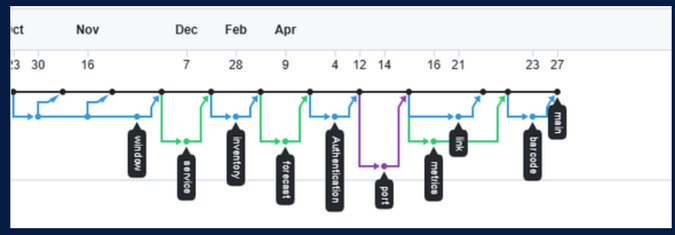
Goals: Deploy all services to a live server accessible over the internet.
Work Completed:
- Provisioned t3.micro EC2 instance on AWS running Ubuntu
- Assigned permanent Elastic IP address
- Configured inbound firewall rules: ports 8001, 8002, 8003 open to 0.0.0.0/0; port 22 restricted to developer IP
- Installed Docker on EC2 using official Docker repository (not apt docker.io)
- Created SmartStock_Deploy repository with docker-compose.yml pulling all three service images from ghcr.io
- Deployed all services with docker compose up -d
- Seeded auth database: docker exec -it smartstock_auth python -m app.seed_users
- Seeded inventory database locally: export DATABASE_URL=... && python seed.py
- Updated frontend BASE_URL values to point to Elastic IP

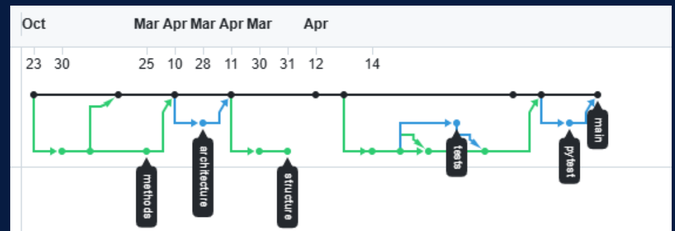
Bugs / Challenges:
- **docker-compose command not found** on newer Ubuntu — the command is docker compose (space, not hyphen) on modern Docker installations
- **Seed script path issue** — docker exec python seed.py failed with "No such file or directory". Fix: docker exec python app/seed.py
- **Prophet memory instability** on t3.micro — managed by avoiding concurrent heavy operations

Goals: Explore scikit-learn Random Forest for stockout risk prediction.
Work Completed:
- Developed Google Colab notebook with full ML pipeline
- Implemented build_features() function extracting four features per item: stock_quantity, avg_daily_usage, days_since_restock, days_until_expiry
- Framed as binary classification: At Risk vs Safe (30-day threshold)
- Trained RandomForestClassifier with 100 estimators
- Evaluated with confusion matrix, classification report, ROC curve, and 5-fold cross-validation
- Connected notebook to live SmartStock API (Option A) to train on real data

Bugs / Challenges:
- **First run on real data: days_since_restock showed zero variance** — all items had the same stocked_date because the seed script hardcoded datetime.now().date() - timedelta(days=200) for every item. The feature correlation matrix showed blank cells for days_since_restock, confirming no signal
- **Fix: updated seed script** — randomised stocked_date between 180 and 365 days ago, and added 1–3 random IN (restock) movements per item spread across the history
- **Re-deployed updated seed script to EC2** — pulled new Docker image after CI built it, ran docker exec python app/seed.py to re-seed
- **Results after fix:** 90% test accuracy, AUC 0.97, feature importances: avg_daily_usage 43.8%, stock_quantity 39.4%, days_since_restock 9.8%, days_until_expiry 7.0%

GitHub History





Problems I Solved

Key challenges encountered during development and how each was resolved through systematic debugging and engineering solutions.

FastAPI Routing

Problem: /scan route matched as /{item_id}, causing 422 errors
Solution: Moved /scan above wildcard route for correct FastAPI matching order

NFC Hardware

Problem: NFC manufacturer library incompatible with Arduino Uno
Solution: Rewired with jumpers and wrote custom SPI register communication code

GUI Threading

Problem: GUI freezing during API calls blocked user interaction
Solution: Moved HTTP requests to background threads; used Tkinter after() to post results safely